PLANT GUARDIAN

GCP Connection and Operations Guide

A consolidated technical note explaining how Plant Guardian uses Google Cloud Platform, how requests and data flow through the system, where each resource is managed, and how to operate the deployment safely.

Project	Value
GCP project	gcp-hackathon-506604
Primary region	asia-south1 (Mumbai)
Application	Plant Guardian full-stack plant care dashboard
Cloud scope	Cloud Run, Cloud SQL, Artifact Registry, Cloud Build, Secret Manager, IAM, Scheduler, Pub/Sub, and Google OAuth

Product message

Most trackers tell you good or bad. Plant Guardian tells you how urgent.

Prepared from the Plant Guardian repository, deployment configuration, and deployed service details supplied for this project.

1. Executive summary

Plant Guardian is deployed as a compact, stateless service platform on Google Cloud. The browser reaches a Next.js frontend on Cloud Run. The frontend uses same-origin /api rewrites to reach the FastAPI backend and Vacation Mode service. The backend owns authentication, plant CRUD, risk calculations, watering history, gamification, pet-safety metadata, and notification orchestration. PostgreSQL data lives in Cloud SQL.

AI text generation is isolated in a separate ai-assistance Cloud Run service. It receives the Groq API key from Secret Manager and is called by vacation-mode through an internal service URL. The design separates UI, API, database, messaging, and AI responsibilities while keeping deployment simple.

Current state

The deployed architecture contains four Cloud Run services, one managed PostgreSQL instance, one regional Artifact Registry repository, Cloud Build image delivery, Secret Manager secrets, six dedicated service accounts, a Cloud Scheduler job, and conditional Pub/Sub notification delivery.

Quick navigation map

Need to do	Console location	Resource
Open live app services	Cloud Run > Services	frontend, backend, vacation-mode, ai-assistance
Inspect database	SQL > Instances	plant-guardian-db
Inspect Docker images	Artifact Registry > Repositories	plant-guardian
Review builds	Cloud Build > History	frontend/backend builds
Manage secrets	Security > Secret Manager	groq-api-key, optional VAPID secret
Review identities	IAM & Admin > Service Accounts / IAM	six Plant Guardian accounts
Inspect schedule	Cloud Scheduler > Jobs	plant-guardian-notifications
Inspect events	Pub/Sub > Topics / Subscriptions	plant-care-notifications
Manage Google login	Google Auth Platform > Clients	Plant Guardian Web client

2. Architecture and request flow

High-level topology

```
Browser
  |
  v
frontend (Next.js on Cloud Run)
  |-- /api/plants, /api/auth, /api/profile, /api/notifications
  |       -> backend (FastAPI on Cloud Run)
  |              |-> Cloud SQL PostgreSQL through Cloud SQL socket
  |              |-> Pub/Sub publisher for asynchronous reminders
  |              |-> browser Web Push when reminders are enabled
  |
  |-- /api/vacation-mode
         -> vacation-mode (FastAPI on Cloud Run)
                -> ai-assistance (FastAPI on Cloud Run)
                       -> Groq API using Secret Manager key

Cloud Scheduler (OIDC identity)
  -> backend /internal/notifications/dispatch
       -> Pub/Sub topic -> authenticated push subscription
```

```
-> backend /internal/notifications/pubsub -> browser Web Push
```

The browser usually sees only the frontend URL. Next.js rewrites keep backend and Vacation Mode URLs server-side. This avoids exposing internal topology in normal browser links and gives the deployment a clean frontend gateway.

Resource identifiers

Identifier	Value	Purpose
Project	gcp-hackathon-506604	Resource, IAM, API, billing, and log boundary.
Region	asia-south1	Primary location for compute and data resources.
Registry	asia-south1-docker.pkg.dev/gcp-hackathon-506604/plant-guardian	Private image repository.
SQL instance	plant-guardian-db	Managed PostgreSQL host.
SQL database	plant_guardian	Application database.

3. GCP services and their role

3.1 Cloud Run: application hosting

Cloud Run runs each Docker container as a managed HTTPS service. It creates revisions for deployments, routes traffic, autos-scales instances, emits logs and metrics, and can scale idle services to zero. Plant Guardian uses four services so the dashboard, core API, vacation planner, and AI adapter remain independently deployable.

Service	What it does	URL
frontend	Next.js dashboard, auth UI, plant cards, profile, reminders, and API proxy.	https://frontend-845145311784.asia-south1.run.app
backend	FastAPI API, sessions, plant logic, PostgreSQL, notifications, and business rules.	https://backend-845145311784.asia-south1.run.app
vacation-mode	Creates vacation watering plans and caretaker briefings.	https://vacation-mode-845145311784.asia-south1.run.app
ai-assistance	Calls Groq for care and vacation wording.	https://ai-assistance-845145311784.asia-south1.run.app

How to access

Console path: Cloud Run > Services > select a service. The service page exposes the URL, revisions, logs, metrics, environment configuration, scaling settings, and traffic split.

```
gcloud run services list --project gcp-hackathon-506604 --region asia-south1
gcloud run services describe backend --project gcp-hackathon-506604 --region asia-south1
gcloud run services logs read backend --project gcp-hackathon-506604 --region asia-south1 --limit 50
```

3.2 Cloud SQL: durable PostgreSQL

Cloud SQL is the production persistence layer. It stores users, sessions, plants, watering rows, care events, notification subscriptions, and application metadata. Risk scores, seasonal frequency, mood, and other care metrics are calculated at response time rather than stored as stale values.

The backend uses a Cloud SQL Unix socket. Its production connection string follows this shape:

```
postgresql+psycopg://USER:PASSWORD@/plant_guardian?host=/cloudsql/PROJECT:REGION:INSTANCE
```

Cloud Run attaches the SQL instance with --add-cloudsql-instances and the backend service account receives roles/cloudsql.client. The backend startup entrypoint runs Alembic migrations before launching FastAPI.

Console path: SQL > Instances > plant-guardian-db. Use the overview, connections, databases, users, backups, operations, logs, maintenance, and Query Insights pages.

```
gcloud sql instances describe plant-guardian-db --project gcp-hackathon-506604
gcloud sql databases list --instance plant-guardian-db --project gcp-hackathon-506604
gcloud sql users list --instance plant-guardian-db --project gcp-hackathon-506604
```

Database protection

Do not delete the Cloud SQL instance or its production data without a backup and an explicit recovery plan. The current deployment script passes the database password through deployment configuration; moving it to Secret Manager is a recommended hardening task.

3.3 Artifact Registry: private image storage

Artifact Registry stores the Docker images executed by Cloud Run. Cloud Build pushes images such as backend:latest and frontend:latest to the regional plant-guardian repository. IAM controls who can push, pull, or delete images.

Console path: Artifact Registry > Repositories > plant-guardian. Review packages, tags, digests, image size, vulnerability findings when enabled, and cleanup policies.

```
gcloud artifacts repositories list --location asia-south1 --project gcp-hackathon-506604
gcloud artifacts docker images list asia-south1-docker.pkg.dev/gcp-hackathon-506604/plant-guardian --include-tags
```

3.4 Cloud Build: reproducible builds

Cloud Build builds the Docker image outside the developer laptop. The submitted source directory is archived, Dockerfile steps run in a managed build worker, and the output image is pushed to Artifact Registry. This makes production builds repeatable.

The frontend build file deploy/cloudbuild-frontend.yaml passes API_URL and VACATION_API_URL for Next.js rewrites. It also passes NEXT_PUBLIC_GOOGLE_CLIENT_ID because that public value is embedded during the Next.js production build.

Console path: Cloud Build > History. Select a build to view substitutions, steps, logs, duration, source, and output image.

```
gcloud builds list --project gcp-hackathon-506604 --limit 20
gcloud builds log BUILD_ID --project gcp-hackathon-506604
```

Build-time behavior

The first build can be slow because it downloads base images and dependencies. For a normal change, submit only .\backend or .\frontend instead of archiving the entire monorepo.

4. Secrets, identities, and security

4.1 Secret Manager

Secret Manager keeps sensitive values out of source code and image layers. The deployment stores the Groq API key as groq-api-key and injects it into ai-assistance as GROQ_API_KEY. If browser push reminders are enabled, the VAPID private key is stored as plant-guardian-vapid-private-key and injected into backend.

The Google Web OAuth client ID is public configuration, not a secret. It belongs in the frontend build argument and backend audience configuration, but a Google client secret must never be placed in a NEXT_PUBLIC_* variable.

Console path: Security > Secret Manager > select a secret. Review versions, replication, IAM, and rotation history. Do not print payload values into logs.

```
gcloud secrets list --project gcp-hackathon-506604
gcloud secrets versions list groq-api-key --project gcp-hackathon-506604
```

4.2 IAM and service accounts

Dedicated service identities reduce blast radius. The deployment creates six accounts and attaches only the permissions needed by each runtime.

Identity	Attached to	Access
plant-guardian-frontend	frontend Cloud Run	No project data role; serves the web app.
plant-guardian-backend	backend Cloud Run	Cloud SQL Client, Pub/Sub publisher, optional VAPID accessor.
plant-guardian-vacation	vacation-mode Cloud Run	Service runtime; calls AI Assistance.
plant-guardian-ai	ai-assistance Cloud Run	Groq secret accessor only.
plant-guardian-scheduler	Cloud Scheduler OIDC	Cloud Run Invoker on backend dispatch.
plant-guardian-pubsub-push	Pub/Sub push OIDC	Cloud Run Invoker on backend worker.

The deployer receives Service Account User on these identities so it can attach them to Cloud Run. The Pub/Sub service agent receives Service Account Token Creator only on the push identity so it can mint an authenticated push token.

Console path: IAM & Admin > Service Accounts for account-level inspection, and IAM & Admin > IAM for project bindings. Prefer managed identities over downloaded service-account keys.

```
gcloud iam service-accounts list --project gcp-hackathon-506604
gcloud projects get-iam-policy gcp-hackathon-506604
```

5. Notifications: Scheduler and Pub/Sub

5.1 Cloud Scheduler

Cloud Scheduler is the time-based trigger. The job plant-guardian-notifications runs every 15 minutes in UTC and sends POST /internal/notifications/dispatch. It uses an OIDC token minted for plant-guardian-scheduler. The backend verifies the token audience and caller identity before scanning due or overdue plants.

Console path: Cloud Scheduler > Jobs > plant-guardian-notifications. Inspect the schedule, time zone, target URI, OIDC service account, retry policy, last run, and response status. Run now is useful for controlled tests.

```
gcloud scheduler jobs describe plant-guardian-notifications \
  --location asia-south1 --project gcp-hackathon-506604
gcloud scheduler jobs run plant-guardian-notifications \
  --location asia-south1 --project gcp-hackathon-506604
```

5.2 Pub/Sub

Pub/Sub is the asynchronous transport. The backend publishes a compact delivery event to plant-care-notifications. The push subscription plant-care-notifications-push calls POST /internal/notifications/pubsub using a Google-signed OIDC token for plant-guardian-pubsub-push. The backend loads plant and browser subscription context from PostgreSQL and performs Web Push delivery.

Pub/Sub provides retry and decoupling. Idempotent delivery records prevent duplicate notifications when Scheduler scans or Pub/Sub pushes are retried. The topic and subscription are created only when both VAPID public and private keys are provided.

Console path: Pub/Sub > Topics > plant-care-notifications, then Pub/Sub > Subscriptions > plant-care-notifications-push. Review push endpoint, authentication identity, retention, retries, and delivery metrics.

```
gcloud pubsub topics describe plant-care-notifications --project gcp-hackathon-506604
gcloud pubsub subscriptions describe plant-care-notifications-push --project gcp-hackathon-506604
```

Status

Pub/Sub is implemented in the repository and deployment script. It is active only when VAPID notification configuration is enabled. If VAPID keys are absent, the rest of Plant Guardian still deploys and the direct/local notification path remains available.

6. Google login and OAuth

Google Sign-In is managed through Google Auth Platform, separate from Cloud Run. The frontend loads Google Identity Services and receives an ID-token credential. The backend validates the token audience against GOOGLE_CLIENT_ID, then starts or creates the Plant Guardian session.

New Google users must complete name, place, and pets before account creation. Existing users can sign in with Google once their email is associated with an account.

Configuration

- Create a Web application OAuth client in Google Auth Platform.
- Register http://localhost:3000 and each deployed frontend origin as Authorized JavaScript origins.
- Use External audience for users outside your organization; while in Testing, add testers under Audience > Test users.
- Keep default openid, email, and profile scopes; no extra Google API scopes are required.
- No redirect URI is needed for the JavaScript callback flow used by this project.

Connection to the deployment

Variable	Where it goes	Why
GOOGLE_CLIENT_ID	Backend Cloud Run environment	Audience used to verify Google ID tokens.
NEXT_PUBLIC_GOOGLE_CLIENT_ID	Frontend Cloud Build argument	Public browser client ID embedded in Next.js.

```
gcloud run services update backend \
  --region asia-south1 \
  --update-env-vars=GOOGLE_CLIENT_ID=YOUR_CLIENT_ID.apps.googleusercontent.com
```

Important

Updating only the backend is not enough. NEXT_PUBLIC_GOOGLE_CLIENT_ID is read during the frontend production build, so changing it requires a new frontend image build and Cloud Run frontend deployment.

7. Deployment and change workflow

Full deployment

deploy/gcp-deploy.sh enables APIs, creates or reuses infrastructure, configures IAM, builds four images, deploys services in dependency order, and optionally creates Pub/Sub and Scheduler resources. It is appropriate for initial setup or a coordinated infrastructure refresh.

```
# Bash / Git Bash
export GCP_PROJECT_ID=gcp-hackathon-506604
export GCP_REGION=asia-south1
export GROQ_API_KEY=YOUR_NEW_KEY
bash deploy/gcp-deploy.sh
```

On Windows CMD use set VAR=value. On PowerShell use $env:VAR = value. Do not use export in CMD or PowerShell, and do not add backslashes before underscores.

Single-service redeploy

For ordinary code changes, build and deploy only the changed service. This is faster and avoids touching unchanged AI, Vacation Mode, database, Scheduler, or Pub/Sub resources.

```
# Backend
gcloud builds submit .\backend --project=%PROJECT_ID% --tag=%BACKEND_IMAGE%
gcloud run deploy backend --project=%PROJECT_ID% --region=%REGION% \
  --image=%BACKEND_IMAGE% \
  --service-account=plant-guardian-backend@%PROJECT_ID%.iam.gserviceaccount.com

# Frontend
gcloud builds submit .\frontend --project=%PROJECT_ID% \
  --config=deploy\cloudbuild-frontend.yaml \
  --substitutions="_API_URL=%BACKEND_URL%,_VACATION_API_URL=%VACATION_URL%,_GOOGLE_CLIENT_ID=%GOOGLE_CLIENT_ID%,_IMAGE=%
gcloud run deploy frontend --project=%PROJECT_ID% --region=%REGION% \
  --image=%FRONTEND_IMAGE% --allow-unauthenticated
```

Revision and rollback

Every Cloud Run deploy creates a revision. In Cloud Run > service > Revisions, route traffic to a known-good revision if a deployment fails. Keep the previous revision until smoke tests pass.

```
gcloud run services update-traffic backend \
  --region asia-south1 \
  --to-revisions=backend-00002-ABC=100
```

8. Operations, cost, and security

Daily checks

- Cloud Run: review startup logs, error rates, latency, instance count, and revision traffic.
- Cloud SQL: confirm backups, storage headroom, connection health, and migration success.
- Cloud Build: retain successful build IDs and inspect failed step logs.
- Scheduler: inspect the last response and use Run now only for controlled tests.
- Pub/Sub: watch undelivered messages, retry counts, and push response codes.
- Secret Manager and IAM: review access bindings and rotate secrets deliberately.

Cost model

Resource	Main cost driver	Practical control
Cloud Run	Request compute and running instances.	Tune memory, concurrency, max instances; allow scale-to-zero.
Cloud SQL	Provisioned instance, storage, backups, network.	Choose the right tier and backup retention.
Cloud Build	Build minutes and artifact storage.	Build only changed services; reuse layers.
Artifact Registry	Stored image layers and retrieval.	Clean old untagged images.
Scheduler / Pub/Sub	Job executions, messages, retention.	Keep one bounded 15-minute workflow.

Security checklist

- Rotate any API key exposed in a terminal transcript, chat, screenshot, or repository.
- Never commit .env, database passwords, Groq keys, VAPID private keys, or service-account keys.
- Use the dedicated service accounts and OIDC callers created by the deployment.
- Restrict CORS_ORIGINS to deployed frontend origins in a hardened production setup.
- Keep notification routes protected; do not make internal endpoints public just to test them.
- Use Cloud SQL backups and verify restore procedures before a production release.
- Use OAuth Testing users until the Google app is ready for its intended audience.

Recommended hardening

Move the Cloud SQL password from deployment environment configuration into Secret Manager and inject it into the backend. This is the main security improvement still visible in the current deployment pattern.

9. Verification checklist and references

Check	Expected result
Cloud Run	Four services are healthy and the intended revisions receive traffic.
Frontend	Dashboard opens, API proxy works, and Google button is interactive.
Backend	GET /health reports database-aware health; CRUD and watering work.
Database	Alembic schema is current and backups are configured.
Google OAuth	Authorized origins and test users are correct; client ID matches backend and frontend.
Scheduler	15-minute job exists with OIDC and a successful last response.
Pub/Sub	Topic and authenticated push subscription exist when VAPID mode is enabled.
Rollback	Previous healthy frontend and backend revisions remain available.

Smoke-test commands

```
gcloud run services list --project gcp-hackathon-506604 --region asia-south1
curl https://backend-845145311784.asia-south1.run.app/health
gcloud scheduler jobs run plant-guardian-notifications --location asia-south1 --project gcp-hackathon-506604
```

Project references

- README.md - overview, environment variables, APIs, local setup, and deployment summary.
- DEPLOY_GCP.md - architecture, IAM matrix, Pub/Sub flow, redeploy, teardown, and cost notes.
- deploy/gcp-deploy.sh - automated GCP resource setup and service deployment.
- deploy/cloudbuild-frontend.yaml - frontend build arguments and image output.
- .env.example and docker-compose.yml - local configuration and service topology.

Official documentation:

Cloud Run

Cloud SQL

Artifact Registry

Cloud Build

Secret Manager

IAM service accounts

Cloud Scheduler

Pub/Sub

Google Identity Services

Final architecture statement

Plant Guardian uses GCP as a managed runtime and integration layer: Cloud Run hosts stateless services, Cloud SQL stores durable state, Artifact Registry and Cloud Build deliver containers, Secret Manager protects runtime secrets, IAM limits access, Scheduler supplies time-based triggers, Pub/Sub supplies durable asynchronous delivery, and Google Auth Platform supplies optional Google account authentication.

10. Troubleshooting quick reference

Symptom	First checks	Likely fix
Cloud Run deploy fails	Cloud Build History and the failed step log.	Fix the image build or environment value, then redeploy the affected service only.
Frontend cannot reach API	Cloud Run frontend logs, API_URL, and Next.js rewrite target.	Confirm the backend URL is HTTPS and that the new frontend revision received traffic.
Database connection error	Cloud Run backend logs, Cloud SQL instance state, and --add-cloudsql-instances attachment.	Restore the attachment, verify DATABASE_URL format, and check IAM Cloud SQL Client access.
Google button does nothing	Browser console, frontend build variable, OAuth client origins, and test-user list.	Rebuild frontend with NEXT_PUBLIC_GOOGLE_CLIENT_ID and add the exact deployed origin.
Reminder not delivered	Scheduler last response, Pub/Sub undelivered messages, and VAPID configuration.	Run the Scheduler job manually, inspect push response codes, and verify the authenticated push identity.
Need to undo a release	Cloud Run Revisions and traffic percentages.	Route 100 percent of traffic to the last known-good revision, then investigate the failed one.

Safe operating rule

When diagnosing production, prefer read-only inspection first: logs, revisions, metrics, IAM policy, Scheduler history, and Pub/Sub delivery status. Change one service or one configuration value at a time, record the revision, and keep a known-good rollback target.